SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Sri Lanka Institute of Information Technology

Faculty of Computing

IT2130 - Operating Systems and System Administration

Dr.Sanvitha Kasthuriarachchi

Year 02 and Semester 02

Lecture 03

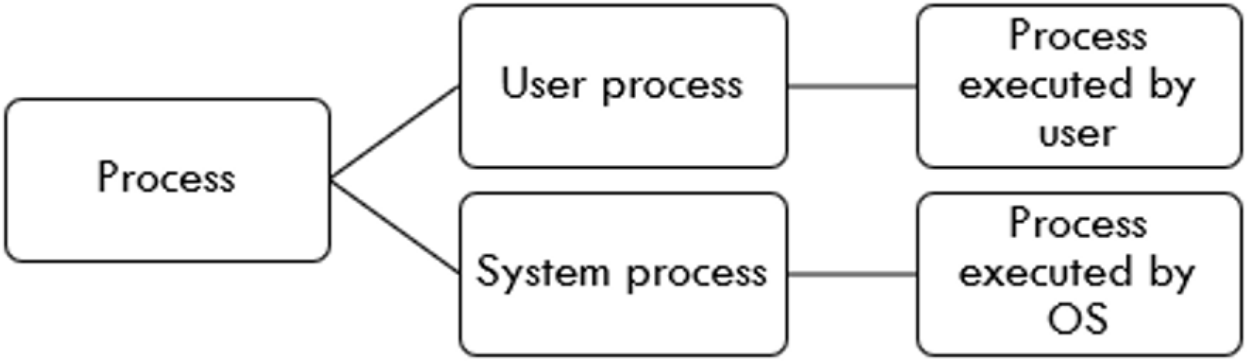
Processes Management in Operating Systems

Chapter 3: Processes

- Process Concept
- Process State
- Process Control Block
- Process Scheduling
- Operations of Processes
- Inter process Communication

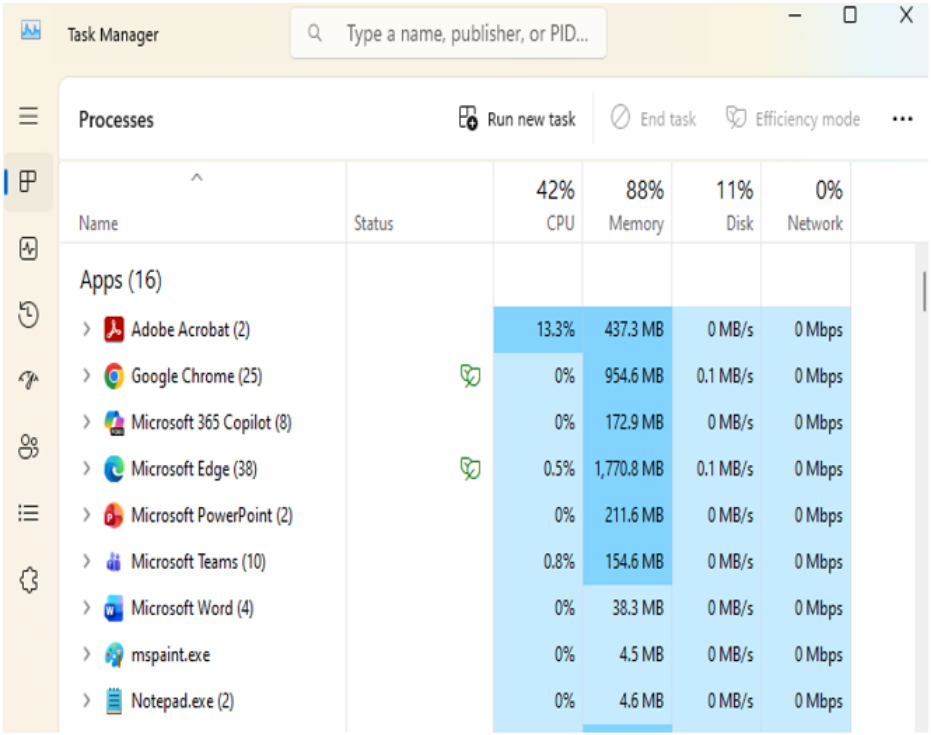
What is a process

- **Program**: a step-by-step instruction set that are going to executed by the CPU. It's a passive entity.
- **Process**: When the program is running in the CPU, it becomes an active entity. This is called a process.



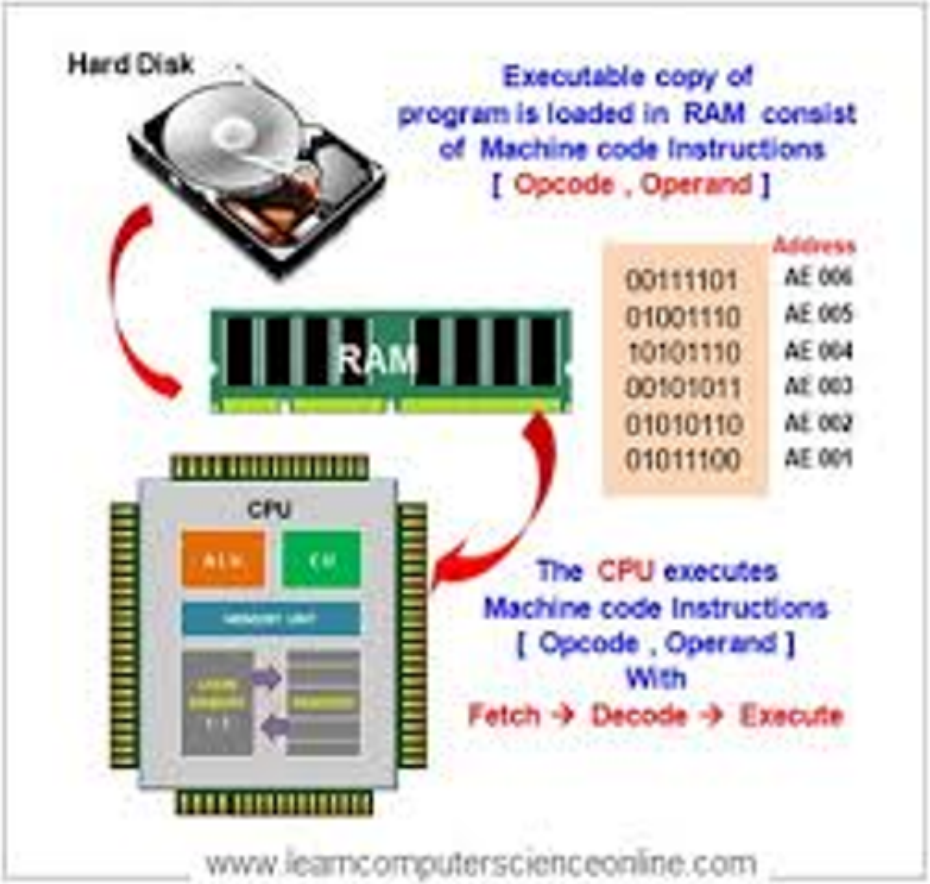
Process Concept

- The Operating System refers to the process as jobs or tasks.
- A process needs certain resources, including CPU time, memory, files, and I/O devices to accomplish its task.



Process Concept

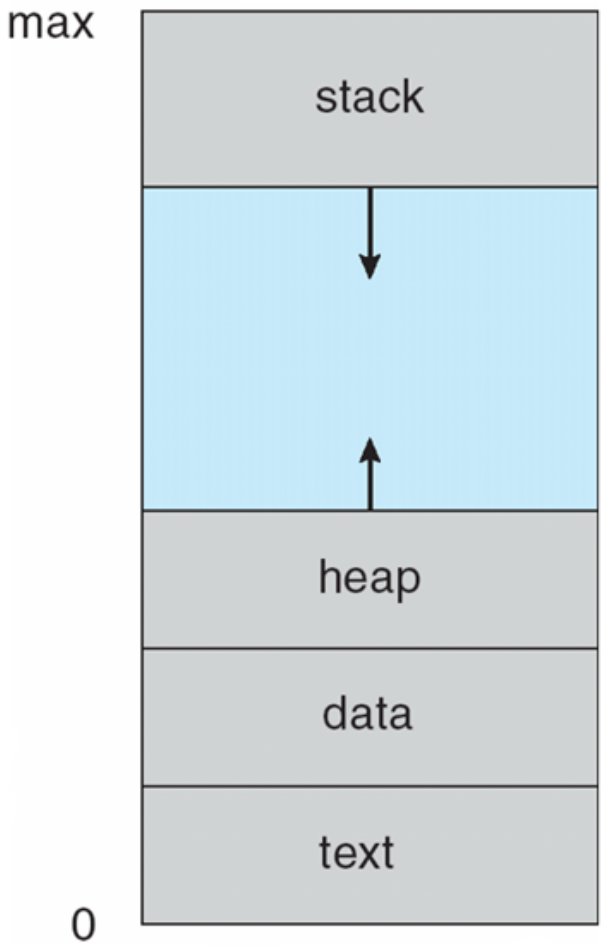
- The process execution must progress in sequential fashion.
- A process has a Program Counter which is a CPU register to specify the next instruction to execute.



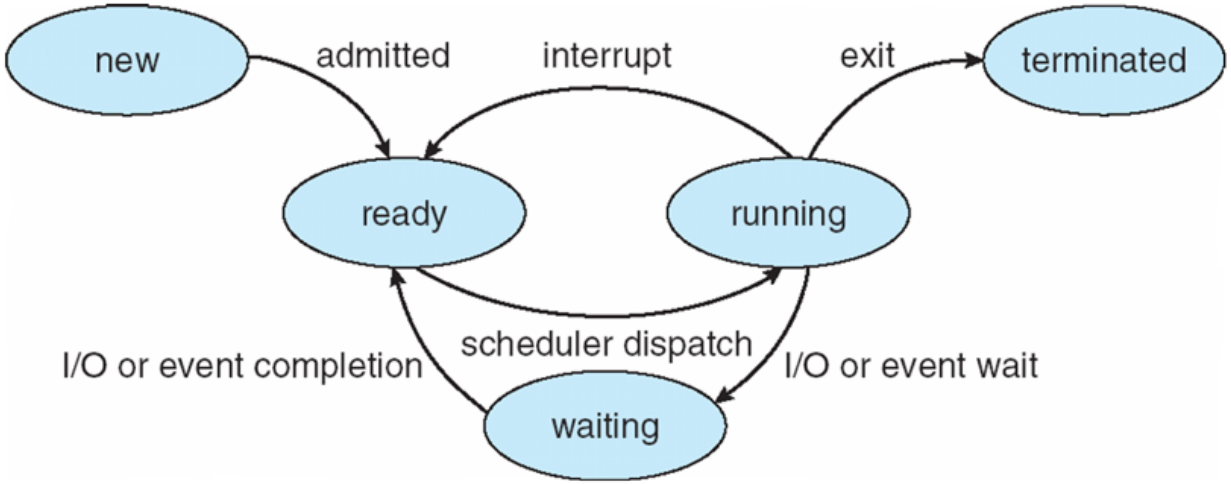
Process Concept

A process consists of multiple parts

- **Text Section** - The program code is here.
- **Stack section** – location for temporary data such as function parameters, return addresses, local variables
- **Data section** - containing global variables
- **Heap section** - containing spare memory dynamically allocated during run time



Process State

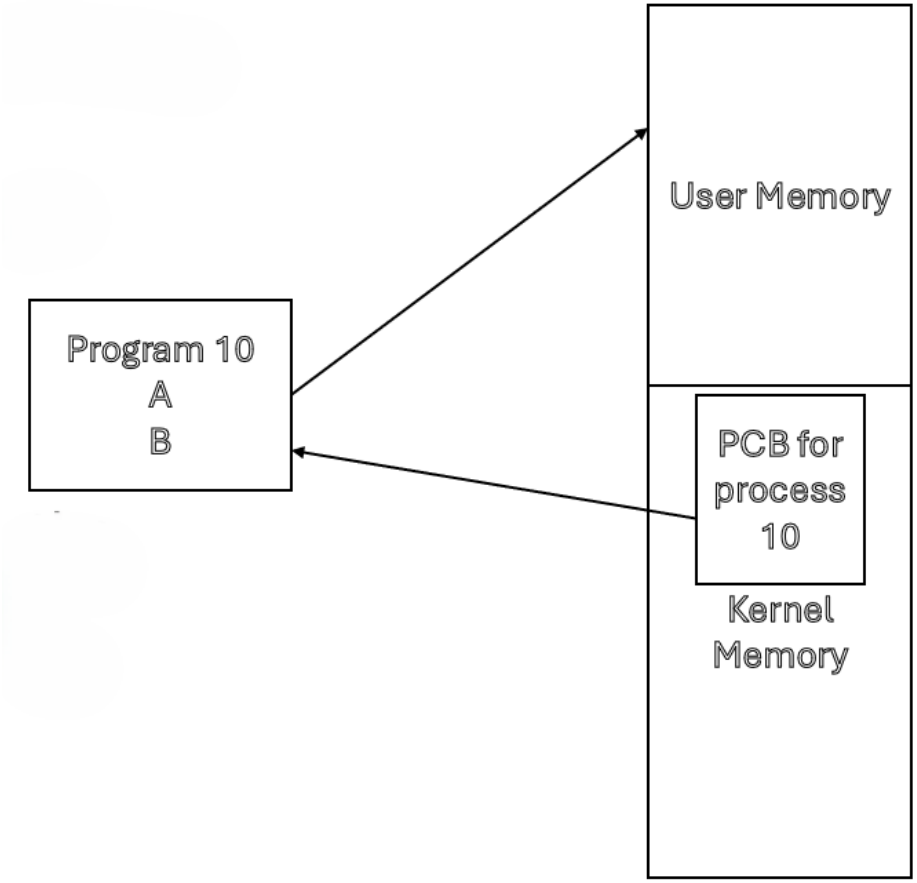


As a process executes, it changes state

- **new**: The process is being created
- **running**: Instructions are being executed
- **waiting**: The process is waiting for some event to occur
- **ready**: The process is waiting to be assigned to a processor
- **terminated**: The process has finished execution

Process Control Block (PCB)

- Every process has a Process Control Block (PCB) to store the essential details about the process.
- When a program loads to the RAM, the program will get the memory from the user memory, and the PCB will get the memory from the kernel memory area.

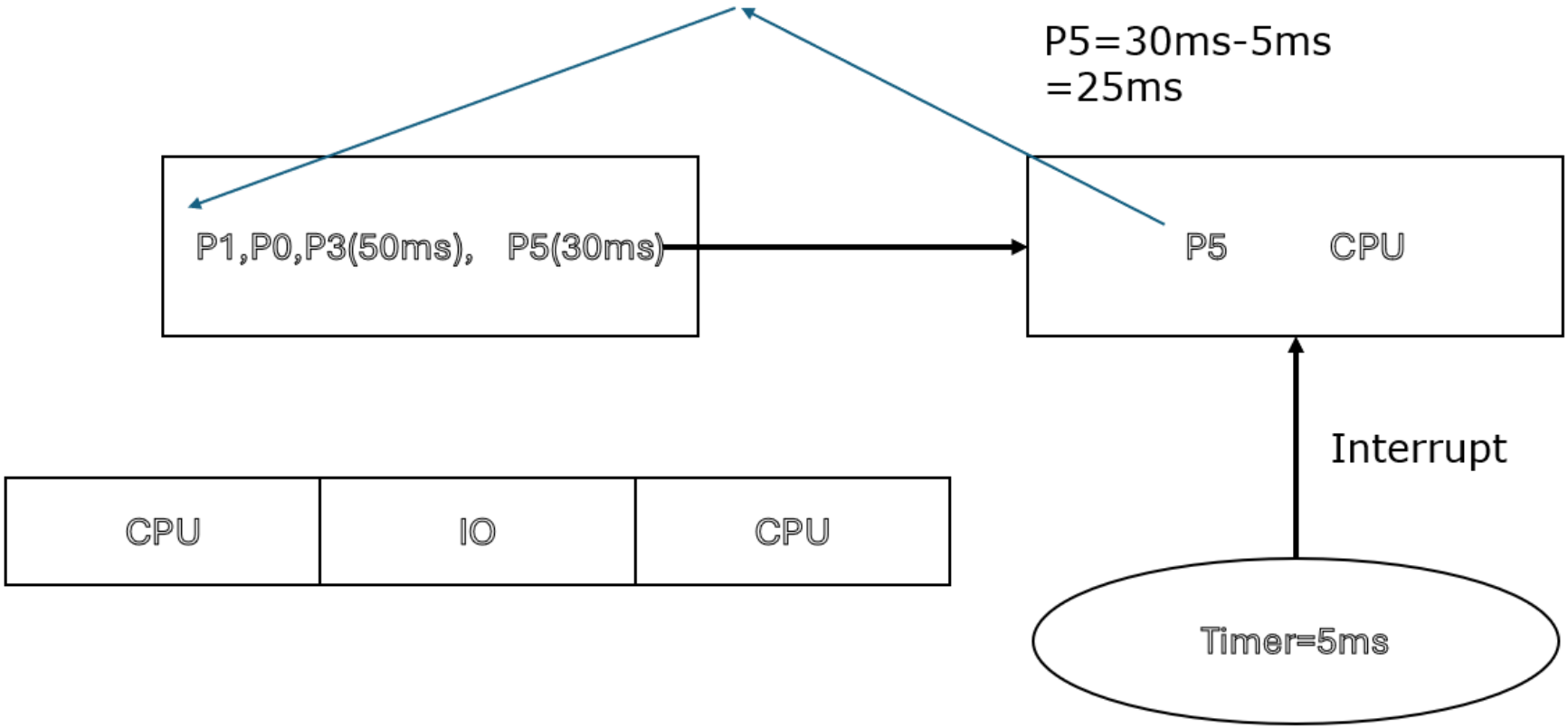


Process Control Block (PCB)

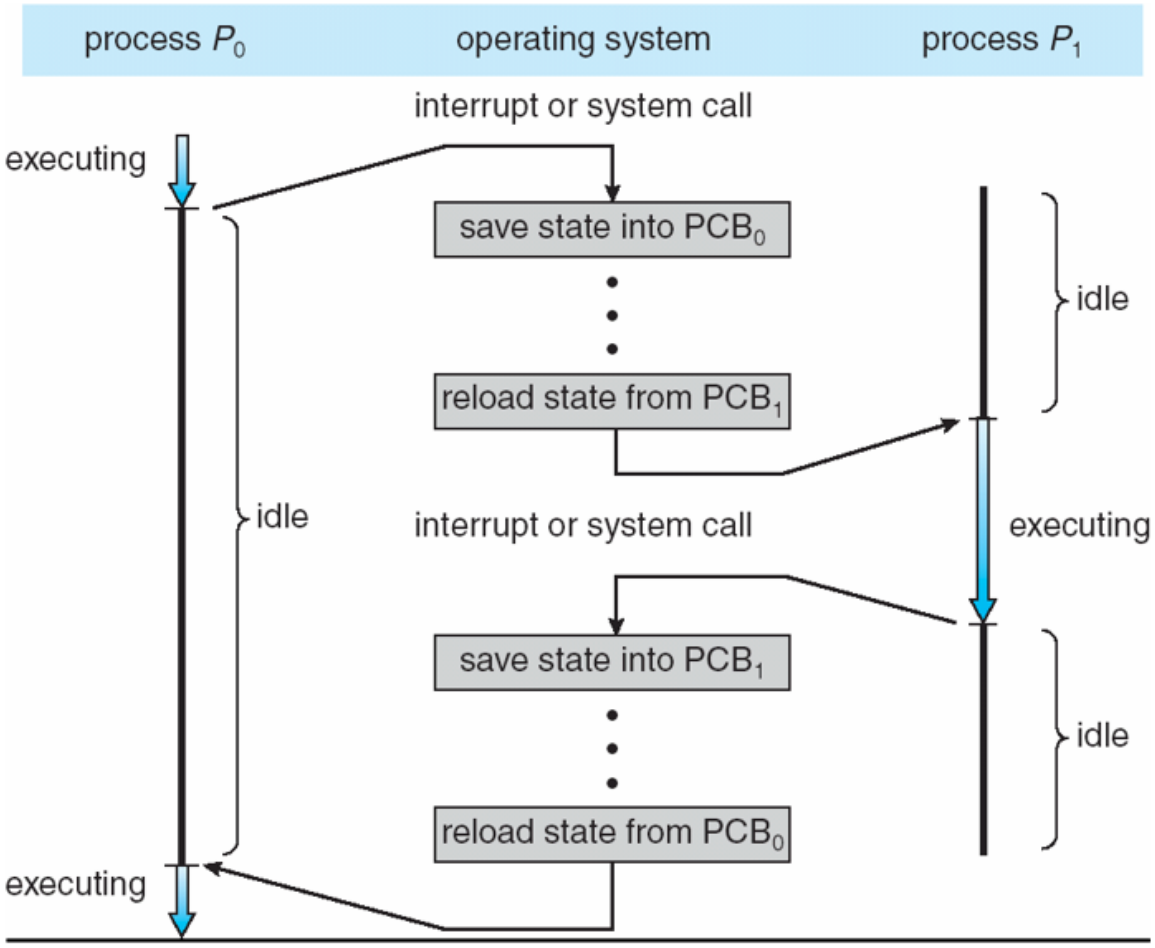
Each PCB contains:

- **Process state** – running, waiting, etc
- **Process number** (process ID)
- **Program counter** – location of instruction to next execute
- **CPU registers** – contents of all process-centric registers
- **CPU scheduling information**- priorities, scheduling queue pointers
- **Memory-management information** – memory allocated to the process
- **Accounting information** – CPU used, clock time elapsed since start, time limits
- **I/O status information** – I/O devices allocated to process, list of open files

Process Execution



CPU Switch From Process to Process



Process Scheduling

- There are schedulers to control the process execution in the system.
- Schedulers in multiprogramming environment maximizes CPU use and in time sharing, it quickly switches processes onto CPU.
- There are two main schedulers.
  - **Short-term scheduler** (or CPU scheduler) – selects which process should be executed next and allocates CPU
  - **Long-term scheduler** (or job scheduler) – selects which processes should be brought into the memory/ ready queue
  - The long-term scheduler controls the degree of multiprogramming

Degree of Multiprogramming

- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- The long-term scheduler controls the degree of multiprogramming and strives for good process mix

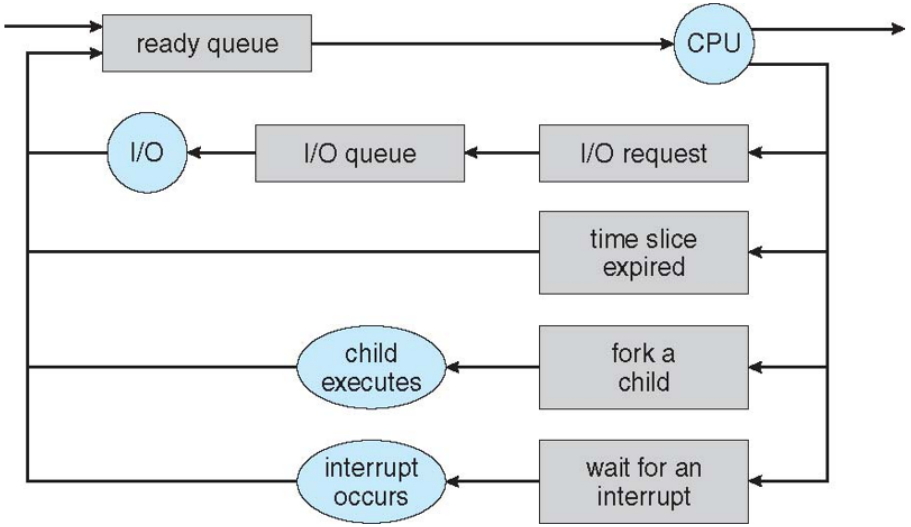
Addition of Medium Term Scheduler

- There are some systems which the Long Term Scheduler is not available.
- They get a Medium-term scheduler to decrease the degree of multi programming.

Process Scheduling with Queues

There are several scheduling queues of processes

- **Job queue** – set of all processes in the system
- **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
- **Device queues** – set of processes waiting for an I/O device
- Processes migrate among the various queues



Context Switch

- Context of a process represented in the PCB.
- When CPU switches from one process to another, the system must save the state of the old process and load the saved state for the new process via a context switch.
- Context-switch time is overhead; the system does no useful work while switching. The more complex the OS and the PCB $\rightarrow$ the longer the context switch.

Operations on Processes

There are three main operations.

- Process Creation
- Process Termination
- Inter Process Communication

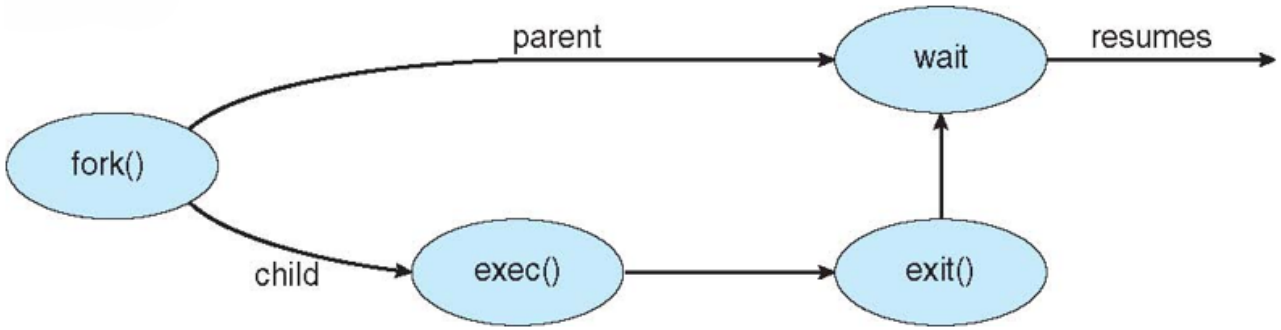
Process Creation

- Parent process creates child processes, which, in turn create other processes, forming a tree of processes
- Generally, process identified and managed via a process identifier (pid)

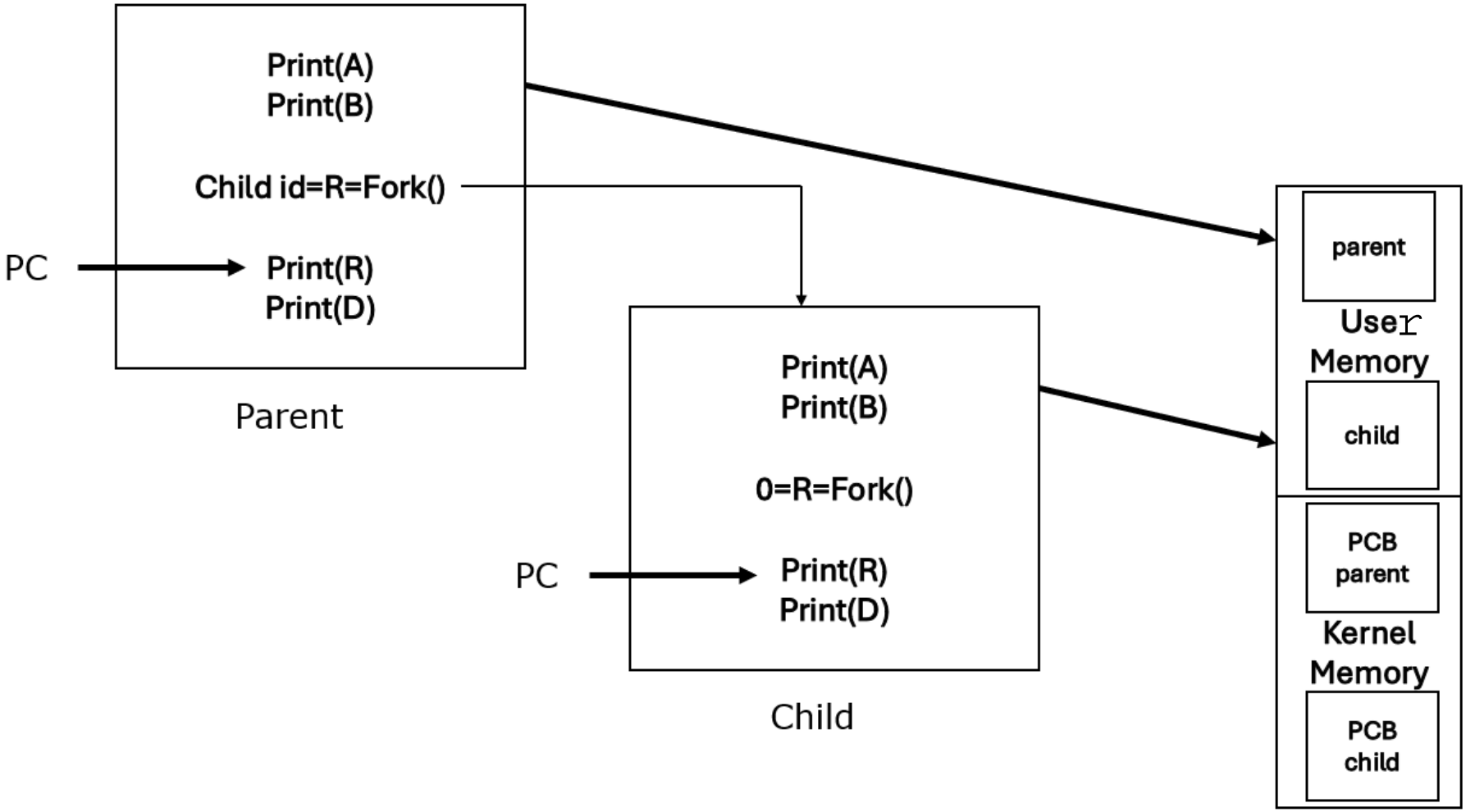
```
systemd─┬─ModemManager───2*[{ModemManager}]
        ├─NetworkManager───2*[{NetworkManager}]
        ├─accounts-daemon───2*[{accounts-daemon}]
        ├─acpid
        ├─avahi-daemon───avahi-daemon
        ├─bluetoothd
        ├─chronyd───chronyd
        ├─colord───2*[{colord}]
        ├─cron
        ├─cups-browsed───2*[{cups-browsed}]
        ├─cupsd───4*[dbus]
        ├─dbus-broker-lau───dbus-broker
        ├─fwupd───4*[{fwupd}]
        ├─gdm3─┬─gdm-session-wor─┬─gdm-x-session─┬─Xorg───22*[{Xorg}]
        │      │                 │               ├─gnome-session-b───2*[{gnome-
        │      │                 │               └─2*[{gdm-x-session}]
        │      │                 └─2*[{gdm-session-wor}]
        │      └─2*[{gdm3}]
        ├─gnome-keyring-d───3*[{gnome-keyring-d}]
        ├─networkd-dispat
        ├─packagekitd───2*[{packagekitd}]
        ├─polkitd───2*[{polkitd}]
        ├─python3
```

Process Creation

- Process runs in the user memory of the RAM duplicates (fork()) and create a child process in the same address space.
- exec() system call used after a fork() to replace the process' memory space with a new program
- Parent and child processes execute concurrently. Always, the parent process waits until children terminates.
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources



Process Creation



Process Termination

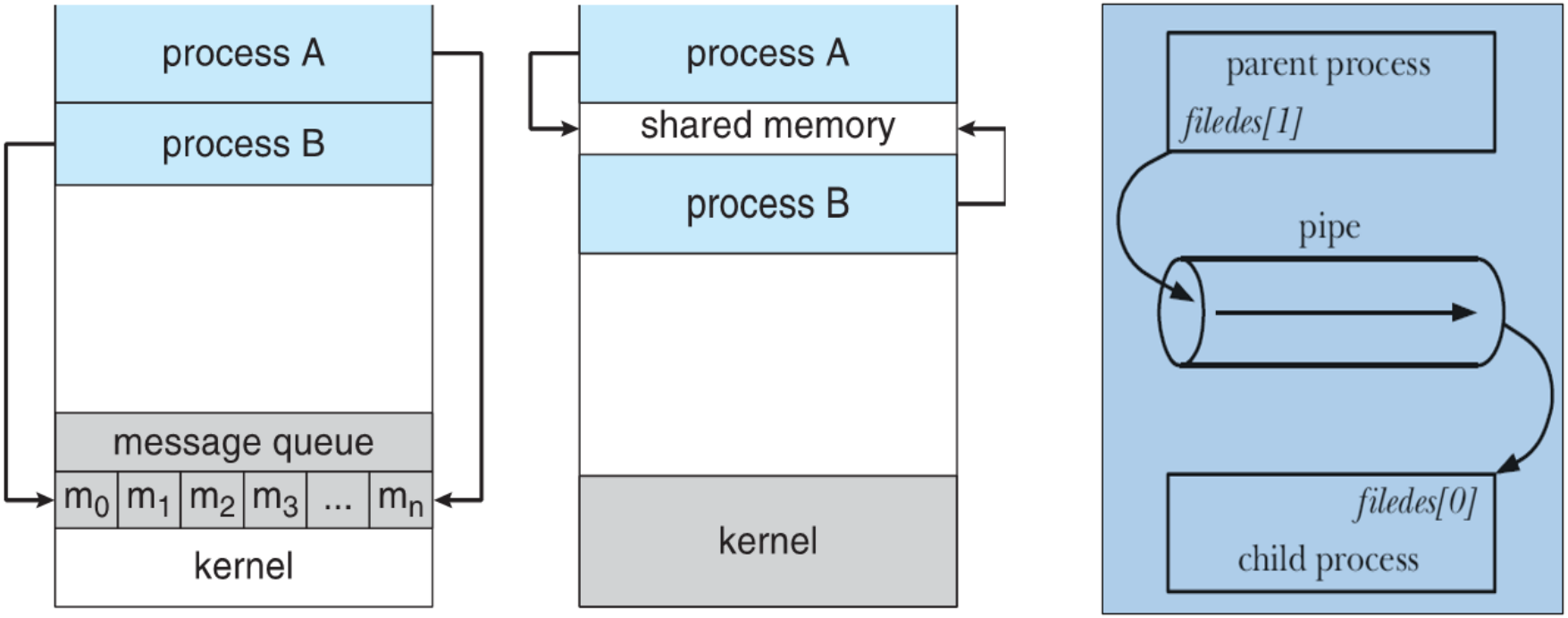
- A process terminates when it finishes executing its final statement and asks the operating system to delete it by using exit() system call.
- At this point, the process may return a state value to its parent process via the wait() system call.
- All the resecures of the process including physical and virtual memory, open files and I/O buffers are deallocated by the operating system.

Inter Process Communication (IPC)

- Processes within a system may be independent or cooperating
- Cooperating process can affect or be affected by other processes, including sharing data.
- Cooperating processes need Inter Process communication (IPC)
- There are three main models of IPC
  - Shared memory
  - Message passing
  - Pipe

Inter Process Communication Models

(a) Message passing (b) shared memory (c) Pipe



(a) (b) (c)

IPC using Message Queues

- There will be a shared message queue buffer in the kernel memory.
- This method enables asynchronous communication, where processes can send messages without waiting for the receiver to be ready, and the messages are stored in the queue until retrieved.
- Message queue does not necessarily implement FIFO every time.

IPC using Message Queues

System calls use for message queues are,

- **msgget()** - create or access a queue
- **msgsnd()** -send a message
- **msgrcv()** - receive a message
- **msgctl()** - control operations like deleting the queue

IPC through Shared Memory

- An IPC method to access a common memory segment by multiple processes.
- All the processes can read and write to the same physical memory.
- Once the shared memory is mapped, there is no Kernal memory involvement required.
- This is the fastest IPC method.

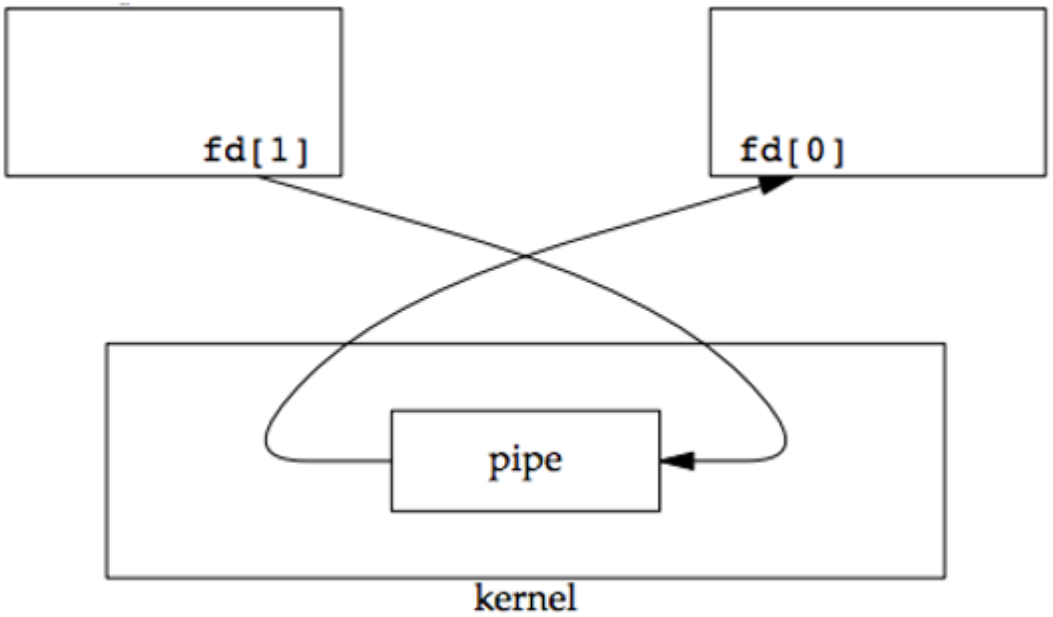
IPC using Shared Memory

System calls use for shared memory are,

- **ftok()** – to generates a unique key to the shared memory segment.
- **shmget()** - to create a new shared memory segment or obtain the ID of an existing one.
- **shmat()**: to attache a shared memory segment to the address space of the calling process.
- **shmdt()**: to detache a previously attached shared memory segment from the calling process's address space.
- **shmctl()**: to control function for shared memory segments.

IPC through PIPE

- In pipe, the output of one process is directed into the input of another process.
- It has an input end and an output end. One can write into a pipe from input end and read from the output end.
- A pipe descriptor, has an array that stores two pointers, one pointer is for its input-end and the other pointer is for its output end.



IPC through PIPE

There are two types of pipes.

- **Anonymous pipes**: The most common type and are used for communication between related processes, such as a parent and its child.
- **Named pipes (FIFOs)**: The more advanced pipe and can be used for communication between unrelated processes.

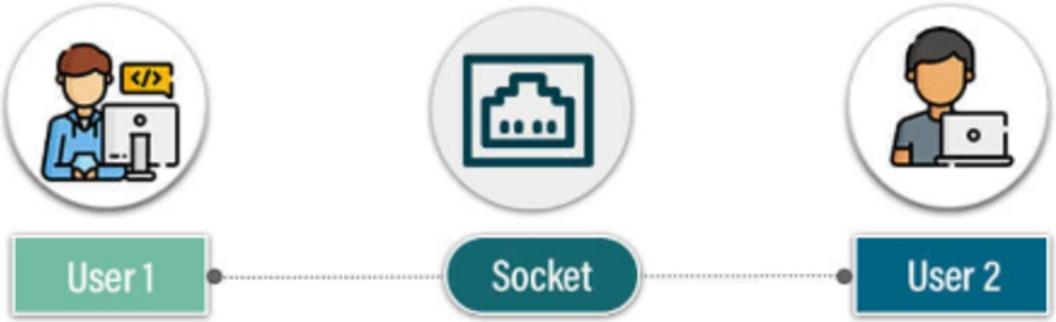
IPC through PIPE

System calls use for pipe are,

- **pipe()** - to create a pipe
- **read()** - read the pipe
- **write()** - write to a pipe

Sockets

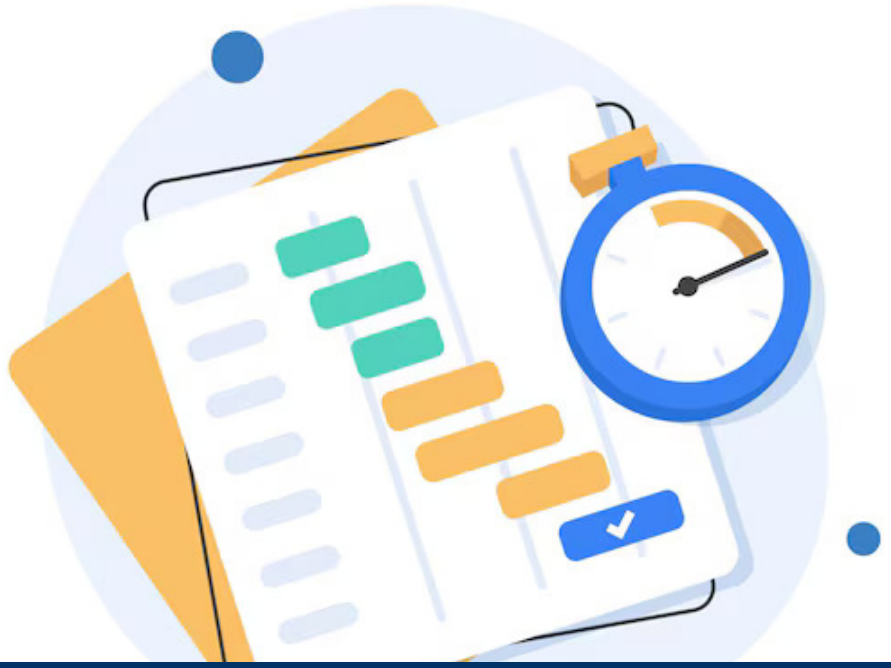
- A socket is defined as an endpoint for communication
- Concatenation of IP address and port – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port 1625 on host **161.25.19.8**
- The communication consists between a pair of sockets



End of Lecture 3

Weekly plan

- **Practical**: Introducing C programming
- **Workshop**:
  - Summary - Processes Management in Operating Systems
  - Discussion of Tutorial 3



THANK
YOU!